

# Deployment of post-quantum cryptography

Simon Abelard

Associate professor, LRE, EPITA (Paris)

Associate researcher, LIP6, Sorbonne (Paris)

Spring 2026

# Short bio

## **Vita:**

- ▶ 5 years in academia  
(PhD, postdoc)
- ▶ 5 years at Thales  
(crypto, cyber)
- ▶ Back in academia

## **Academic activities:**

- ▶ Computer algebra,  
algorithms, complexity
- ▶ Algebraic cryptanalysis  
(multivariate)
- ▶ Protocols (both hybrid  
& quantum)

# Short bio

## **Vita:**

- ▶ 5 years in academia (PhD, postdoc)
- ▶ 5 years at Thales (crypto, cyber)
- ▶ Back in academia

## **Academic activities:**

- ▶ Computer algebra, algorithms, complexity
- ▶ Algebraic cryptanalysis (multivariate)
- ▶ Protocols (both hybrid & quantum)

## **Deployment of PQ crypto:**

- ▶ Prototyping algorithms (software and hardware)
- ▶ Hardening implementations vs side-channels
- ▶ Designing secure protocols for space systems (satellites and ground to space link)
- ▶ Defining security architecture for complex systems
- ▶ Proposing new primitives (Broadcast Encryption)

# Course outline

- ▶ **Generalities:**

- ▶ Hard problems & quantum computers
- ▶ Overview of post-quantum standards

- ▶ **Post-quantum cryptosystems:**

- ▶ Lattice-based cryptography
- ▶ Code-based cryptography

- ▶ **Side-channel attacks:**

- ▶ Examples on post-quantum standards
- ▶ Countermeasures: masking & shuffling

# A brief introduction to post-quantum cryptography

# Cryptology

Broad meaning: protecting digital information against hostile actions. (As opposed to error-correcting codes protecting against noise/errors)

Cryptology can be divided in many topics:

## **Cryptography** (defensive)

- ▶ Symmetric cryptography
- ▶ Asymmetric cryptography
- ▶ Protocols (ZK, TLS, etc.)
- ▶ Homomorphic encryption

## **Cryptanalysis** (offensive)

- ▶ Algorithmic cryptanalysis
- ▶ Side-channel attacks
- ▶ Bug Exploit
- ▶ Social Engineering

# Why study cryptography ?

The weakest link is almost always the user and almost never the cryptographic algorithms, BUT

- ▶ Not studying cryptography is taking the risk to make it the weakest link!
- ▶ A cryptographic failure is less likely but can have a global (and catastrophic) impact

# Why study cryptography ?

The weakest link is almost always the user and almost never the cryptographic algorithms, BUT

- ▶ Not studying cryptography is taking the risk to make it the weakest link!
- ▶ A cryptographic failure is less likely but can have a global (and catastrophic) impact

**Examples:** Playstation network down for a few days (bad implementation of cryptography), LOGJAM attack targeting **8% of the Internet** (poorly dimensioned cryptography), breaking ENIGMA and Japanese variants during WW2 (misuse and bad dimensioning)



# A brief history of symmetric cryptography

Cryptography is old but modern cryptography is  $< 60$  years old!

- ▶ From Antiquity to 20<sup>th</sup> Century: steganography, substitutions and permutation of (groups of) letters.  
Example: famous Cesar cipher, Vigenère cipher, etc.
- ▶ Mechanical devices (Enigma, 1919)  $\rightsquigarrow$  more complex substitutions and cryptanalysis
- ▶ Modern symmetric algorithms: DES (1975), **AES (2000)**

# A brief history of symmetric cryptography

Cryptography is old but modern cryptography is  $< 60$  years old!

- ▶ From Antiquity to 20<sup>th</sup> Century: steganography, substitutions and permutation of (groups of) letters.  
Example: famous Cesar cipher, Vigenère cipher, etc.
- ▶ Mechanical devices (Enigma, 1919)  $\rightsquigarrow$  more complex substitutions and cryptanalysis
- ▶ Modern symmetric algorithms: DES (1975), **AES (2000)**

**Symmetric:** both parties need to have the same **secret** key.

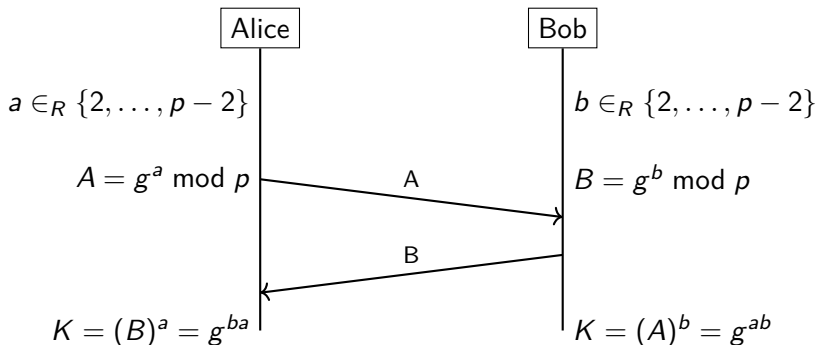
- ▶ How do we transmit secret keys?
- ▶ In a network of  $n$  people, need  $O(n^2)$  secret keys

Acceptable if only a few people (generals and diplomats) use cryptography, with secret keys (code-books) distributed in person.

## Breakthrough: public-key cryptography (1976)

In 1976, Diffie and Hellman introduced **public key cryptography**. PKC is a major breakthrough: no secure internet without it!

Public parameters:  $g, p$



**Security:** is it hard for an attacker to recover  $a$  from  $A$ ?

# Security of public-key cryptography

**Hard problems:** PKC relies on one-way functions based on allegedly hard mathematical problems such as the discrete logarithm and factorization.

**Computational security:** our definition of hard is "that takes an unreasonable amount of time and resource".

# Security of public-key cryptography

**Hard problems:** PKC relies on one-way functions based on allegedly hard mathematical problems such as the discrete logarithm and factorization.

**Computational security:** our definition of hard is "that takes an unreasonable amount of time and resource".

## One-wayness in practice

In 1s, a laptop from 2015 can either perform

- ▶ the product of two 30-million-digit integers
- ▶ the factorization of a 42-digit integer

As of today, nobody ever factored a number with  $> 260$  decimal digits. Previous record (RSA-250) took 2700 core-years.

**Currents standards use numbers of  $> 700$  digits.**

# Quantum computing

The notion of hardness depends on the tools at hand (hardware and algorithms). Quantum computing  $\rightsquigarrow$  new tools!

Some quantum algorithms offer limited speed-up (Grover, 1996), some offer exponential speed-up (Shor, 1994).

## Cryptographic consequences:

- ▶ Symmetric crypto: theoretically safe after doubling key size.  
NIST considers Grover's algorithm non practical.  
NIST: AES-128 is OK vs NÚKIB: switch to AES-256
- ▶ Public-key crypto: both RSA and (EC)DH will be broken by a sufficiently large quantum computers.

# Post-quantum cryptography

Prior to 2015 the quantum threat was not taken seriously.

## **Step 1: the NIST competition (2016)**

- ▶ More than 70 candidates (encryption & signatures)
- ▶ First standards in 2024 (FIPS 203, 204, 205)
- ▶ **Still ongoing** (additional candidates for diversity)

## **Step 2: post-quantum transition (2026)**

- ▶ US rules: post-quantum crypto mandatory by 2030-35
- ▶ EU roadmap: plan by 2026, switch by 2030 (sensitive applications) or by 2035 (everywhere possible)
- ▶ **Main fear:** “store now, decrypt later” attacks targeting long-term secrets or devices (satellites)

# A change of paradigm

**Warning:** Successful transition to post-quantum does not limit to the replacement of cryptographic algorithms!

**No one-size-fits-all solution:** trade-off between

- ▶ Performance (speed, bandwidth, hardware constraints)
- ▶ Resilience to side-channel, ease of use (implementation)
- ▶ Maturity and confidence



# A change of paradigm

**Warning:** Successful transition to post-quantum does not limit to the replacement of cryptographic algorithms!

**No one-size-fits-all solution:** trade-off between

- ▶ Performance (speed, bandwidth, hardware constraints)
- ▶ Resilience to side-channel, ease of use (implementation)
- ▶ Maturity and confidence

**Political challenge:**

- ▶ Realize a complete inventory of cryptography (not easy!)
- ▶ Identify use-cases to prioritize (confidentiality > authenticity)
- ▶ For each use-case, select a suitable post-quantum replacement

# Which post-quantum algorithm should I use?

**General purpose:** structured lattices

- ▶ Kyber aka ML-KEM (NIST FIPS 203) for encryption
- ▶ Dilithium aka ML-DSA (NIST FIPS 204) for signature

**Conservative / back up options:**

- ▶ HQC (code-based) to be standardised by NIST, competitive  
**Warning:** not (yet?) approved by NÚKIB!
- ▶ FrodoKEM (unstructured lattices), less competitive but simple to implement, supported by BSI, ANSSI, and NÚKIB
- ▶ SPHINCS+ aka SLH-DSA (hash-based), NIST FIPS 205, huge but very conservative

# Which post-quantum algorithm should I use?

## **General purpose:** structured lattices

- ▶ Kyber aka ML-KEM (NIST FIPS 203) for encryption
- ▶ Dilithium aka ML-DSA (NIST FIPS 204) for signature

## **Conservative / back up options:**

- ▶ HQC (code-based) to be standardised by NIST, competitive  
**Warning:** not (yet?) approved by NÚKIB!
- ▶ FrodoKEM (unstructured lattices), less competitive but simple to implement, supported by BSI, ANSSI, and NÚKIB
- ▶ SPHINCS+ aka SLH-DSA (hash-based), NIST FIPS 205, huge but very conservative

## **Specific use cases:**

- ▶ Falcon (future FIPS 206) when bandwidth is limited, but implementation / side-channel issues
- ▶ XMSS (hash-based) for software/firmware signing

# Main threats to the post-quantum transition

- ▶ Store now and decrypt later attacks (targets classical crypto and any faulty post-quantum algorithm/implementation). No possible remediation once harm is done.
- ▶ Lack of scientific maturity: two promising NIST candidates (one was a finalist) were broken in a few days with **classical** attacks.
- ▶ Alarming lack of quantum attacks: huge cryptanalytic effort done since 2015 but  $> 90\%$  of it is classical. Yet the NIST competition is about resisting to quantum computing...

# Main threats to the post-quantum transition

- ▶ Store now and decrypt later attacks (targets classical crypto and any faulty post-quantum algorithm/implementation). No possible remediation once harm is done.
- ▶ Lack of scientific maturity: two promising NIST candidates (one was a finalist) were broken in a few days with **classical** attacks.
- ▶ Alarming lack of quantum attacks: huge cryptanalytic effort done since 2015 but  $> 90\%$  of it is classical. Yet the NIST competition is about resisting to quantum computing...

## Possible mitigation:

- ▶ Hybrid classical + post-quantum (only works for a time)
- ▶ Crypto-agility to allow on-the-fly patches
- ▶ Look for non-computational guarantees: quantum & post-quantum hybrids

# Deeper dive into post-quantum primitives

# Quantum-hard problems

## Security of asymmetric cryptography

Remember: we need **hard mathematical problems** that also resist quantum computers.

### Examples:

- ▶ Finding short/close vectors in lattices
- ▶ Decoding random error-correcting codes
- ▶ Inverting hash functions
- ▶ Solving quadratic equations in many variables
- ▶ And many more...

# Foundation of code-based cryptography

## Quick recall:

- ▶ Hard problem  $\rightsquigarrow$  asymmetric cryptography.
- ▶ Shannon's theorem  $\rightsquigarrow$  we can generally decode.
- ▶ Shannon's theorem does not say that it is easy!

## Bounded-distance decoding (BDD)

Given as input  $C \subset \mathbb{F}_q^n$ ,  $y \in \mathbb{F}_q^n$ ,  $0 \leq t \leq n$ ,

Determine whether there is  $c \in C$  s.t.  $d_H(y, c) \leq t$ . ( $y$  close to  $c$ )

## Berlekamp, McEliece, van Tilbørg, 1978

It is (in general) hard to compute  $c$  from  $y$ , i.e. **decoding is hard**.



# McEliece's cryptosystem

Public key  $(G, t)$

Message  $m$



Alice



Bob

Private key

Decode

$G$  generates the code,  $t$  the error weight

# McEliece's cryptosystem

Public key  $(G, t)$

Message  $m$

$e$  of weight  $t$



Alice



Bob

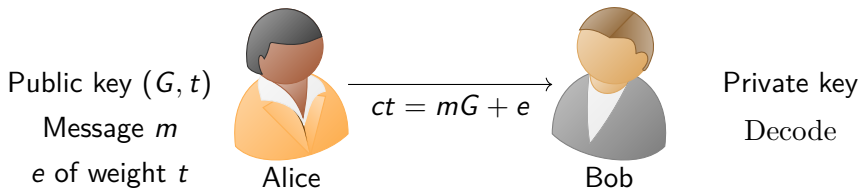
Private key

Decode

$G$  generates the code,  $t$  the error weight

Encryption = add noise to  $mG$  (codeword)  $\rightsquigarrow$  decryption = decoding

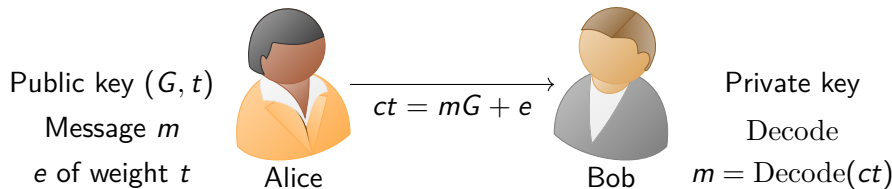
# McEliece's cryptosystem



$G$  generates the code,  $t$  the error weight

Encryption = add noise to  $mG$  (codeword)  $\rightsquigarrow$  decryption = decoding

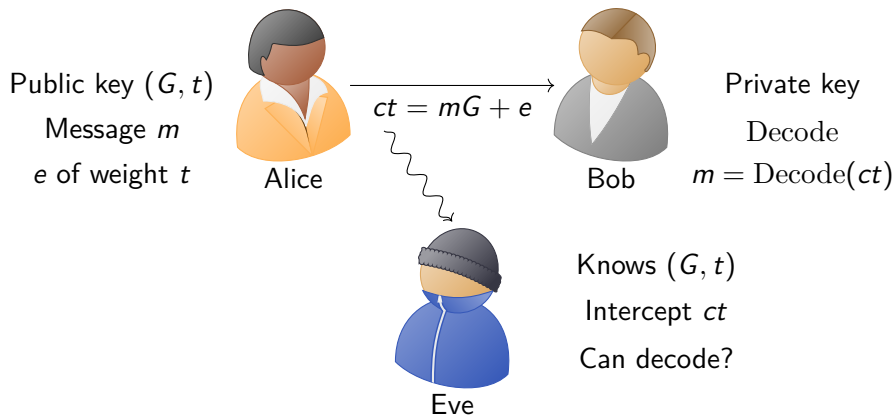
# McEliece's cryptosystem



$G$  generates the code,  $t$  the error weight

Encryption = add noise to  $mG$  (codeword)  $\rightsquigarrow$  decryption = decoding

# McEliece's cryptosystem



$G$  generates the code,  $t$  the error weight

Encryption = add noise to  $mG$  (codeword)  $\rightsquigarrow$  decryption = decoding

Hardness of decoding  $\Rightarrow$  only Bob can recover  $m$

# More than a cryptosystem, a framework

## Parameter choice

Each pair (code, distance) brings a new cryptosystem

This choice is essential to ensure both performance and security!

- ▶ Broken codes: Reed-Solomon, BCH, low-density quasi-cyclic
- ▶ NIST finalist: Goppa codes (Classic McEliece)
- ▶ NIST standard: QC-MDPC (HQC = Hamming Quasi Cyclic)

## Codes in the NIST competition

Classic McEliece: small cipher (128 B), huge pub. key ( $> 200$  kB).

BIKE & HQC: slightly worse than lattices but good options.

Wave: short signature but huge public key ( $> 3$  MB).

# Quasi cyclic Moderate density (QC-MDPC) codes

**Main drawback of code-based:** public key is a huge matrix.

**Circulant matrix:** a matrix described by one vector  $(a_0, \dots, a_{n-1})$ .

$$\begin{pmatrix} a_0 & a_1 & \dots & a_{n-1} \\ a_1 & a_2 & \dots & a_0 \\ \vdots & \vdots & & \vdots \\ a_{n-1} & a_0 & \dots & a_{n-2} \end{pmatrix}$$

**Quasi-cyclic codes:** codes described by blocks of circulant matrices  $\rightsquigarrow$  public key + ciphertext = a few vectors.

**Moderate density:** each row of the parity-check matrix has  $n$  entries of which only  $\sqrt{n}$  are non-zero.

**Does this decrease security?** So far, not really (there are attacks providing a gain sub-linear in  $n$ ).

## From circulant matrices to polynomials

Consider  $P = a_0 + a_1X + \cdots a_{n-1}X^{n-1}$  modulo  $X^n - 1$ , then

$$X \times P = a_{n-1} + a_0X + a_1X^2 + \cdots + a_{n-2}X^{n-1}.$$

Circulant matrix = represent the  $X^i \times P$  on basis  $1, X, \dots, X^{n-1}$ .

**An isomorphism:** the ring of  $n \times n$  circulant matrices over  $\mathbb{F}$  is isomorphic to  $\mathbb{F}[X]/(X^n - 1)$ .

**Transposition:** we can define  $P^T = a_0 + a_{n-1}X + \cdots a_1X^{n-1}$  so that the above isomorphism commutes with transposition.



# From circulant matrices to polynomials

Consider  $P = a_0 + a_1X + \cdots a_{n-1}X^{n-1}$  modulo  $X^n - 1$ , then

$$X \times P = a_{n-1} + a_0X + a_1X^2 + \cdots + a_{n-2}X^{n-1}.$$

Circulant matrix = represent the  $X^i \times P$  on basis  $1, X, \dots, X^{n-1}$ .

**An isomorphism:** the ring of  $n \times n$  circulant matrices over  $\mathbb{F}$  is isomorphic to  $\mathbb{F}[X]/(X^n - 1)$ .

**Transposition:** we can define  $P^T = a_0 + a_{n-1}X + \cdots a_1X^{n-1}$  so that the above isomorphism commutes with transposition.

**Conclusion:** operations on vectors and circulant matrices become operations on polynomials modulo  $(X^n - 1)$ .

**Parameter choice:** for performance and security, work in  $\mathbb{F}_2[X]/(X^n - 1)$  with  $n$  prime and 2 primitive modulo  $n$ .  
(i.e.  $2^k = 1 \bmod n$  iff  $(n - 1)$  divides  $k$ )

# The KEM BIKE (Blt-flipping Key Encapsulation)

**Private key:**  $h_0, h_1 \in \mathbb{F}_2[X]/(X^n - 1)$  of Hamming weight  $w$ .

**Public key:**  $h = h_1 h_0^{-1}$  in  $\mathbb{F}_2[X]/(X^n - 1)$ .

**Encryption:** message  $m = (e_0, e_1)$  both with weight  $\leq t$ .

Ciphertext is  $c = e_0 + e_1 h$ .

**Decryption:** first, compute  $h_0 c = h_0 e_0 + e_1 h_1$ .

Then  $\text{Decode}(h_0 c, h_0, h_1, t)$  yields  $m = (e_0, e_1)$ .

# The KEM BIKE (Blt-flipping Key Encapsulation)

**Private key:**  $h_0, h_1 \in \mathbb{F}_2[X]/(X^n - 1)$  of Hamming weight  $w$ .

**Public key:**  $h = h_1 h_0^{-1}$  in  $\mathbb{F}_2[X]/(X^n - 1)$ .

**Encryption:** message  $m = (e_0, e_1)$  both with weight  $\leq t$ .

Ciphertext is  $c = e_0 + e_1 h$ .

**Decryption:** first, compute  $h_0 c = h_0 e_0 + e_1 h_1$ .

Then  $\text{Decode}(h_0 c, h_0, h_1, t)$  yields  $m = (e_0, e_1)$ .

Why?  $(h_0, h_1) \rightsquigarrow$  quasi-circulant matrix  $H = (H_0 \ H_1)$ .

Then  $h c_0 = (e_0, e_1) H^T$  is called a **syndrome** of  $e$ .

**Decoder:** (syndrome, matrix  $H$ )  $\mapsto$  error  $(e_0, e_1)$ .

## Remarks:

- ▶ Decoding is hard if  $h_0, h_1$  are unknown
- ▶ Here the message is the error (Niederreiter vs McEliece)

# The KEM HQC (Hamming Quasi-Cyclic)

## Similarities with BIKE:

Keys:  $sk = (x, y)$  of weight  $w$ ,  $pk = x + hy$  with  $h$  random.

**Differences:** McEliece framework + another layer (code  $\mathcal{C}$ ).

**Encrypt:**  $ct = (u, v)$ ,  $u = r_1 + hr_2$ ,  $v = \text{Encode}_{\mathcal{C}}(m) + pkr_2 + e$ .

**Decrypt:** Compute  $v - uy = \text{Encode}_{\mathcal{C}}(m) + xr_2 + e - r_1y$ .

If  $xr_2 + e - r_1y$  has small weight,  $\text{Decode}_{\mathcal{C}}(v - uy) = m$ .

## Concept of HQC:

- ▶ Someone knowing  $(x, y)$  will only have to decode  $\mathcal{C}$ .
- ▶ Anyone else must decode the QC-MDPC before.
- ▶ The code  $\mathcal{C}$  need not be hard to decode!

## Decoding methods

### **For BIKE: BGF (black gray flip decoder)**

From arbitrary  $e = (e_0, e_1)$ , look at equation  $hc_0 = eH^T$ .

Identify bits of  $e$  causing  $hc_0 \neq eH^T$ , flip them and repeat.

Conceptually simple decoder but hard to analyze (security issue).

**For HQC:** decoding is done on  $\mathcal{C}$  which is made of “easy” codes (Reed-Solomon and Reed-Muller), hence no need to decode the quasi-cyclic part.

# Decoding methods

## **For BIKE: BGF (black gray flip decoder)**

From arbitrary  $e = (e_0, e_1)$ , look at equation  $hc_0 = eH^T$ .

Identify bits of  $e$  causing  $hc_0 \neq eH^T$ , flip them and repeat.

Conceptually simple decoder but hard to analyze (security issue).

**For HQC:** decoding is done on  $\mathcal{C}$  which is made of “easy” codes (Reed-Solomon and Reed-Muller), hence no need to decode the quasi-cyclic part.

**Importance in design:** while BIKE is conceptually simpler, it is much easier to model the decoding of HQC and thus predict the decoding failures and their impact on security.

**Differences in performance:** BIKE use more bandwidth but it is faster than HQC.

# Lattice-based cryptography

## Lattices

Given a ring  $R$ , a lattice  $L$  over  $R^n$  is the set of integer linear combinations of  $m$  linearly independent vectors  $b_1, \dots, b_m$ .

$$L = L(b_1, \dots, b_m) = \left\{ \sum_{i=1}^m \alpha_i b_i \mid (\alpha_1, \dots, \alpha_m) \in \mathbb{Z}^m \right\}$$

**Base ring:** most common are  $R = \mathbb{Z}_q$  or  $R = \mathbb{Z}_q[X]/(X^n + 1)$ .

**Definitions:**  $n$  is the *dimension* and  $m$  the *rank* of  $L$ .

**Basis:**  $B = (b_1, \dots, b_m)$  is called a *basis* of  $L$ .

We will see that not all bases are equally convenient: a good basis usually have vectors  $b_i$  of same size and orthogonal.

# Difficult problems on lattices

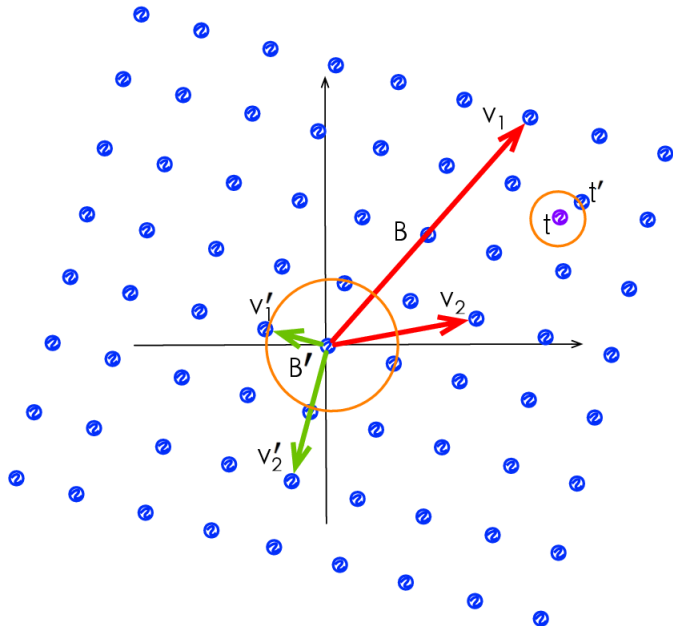
- ▶ **SVP (Shortest vector problem):** find shortest  $v \in L \setminus \{0\}$
- ▶ **CVP (Closest vector problem):** given  $t \notin L$ , find  $v \in L$  the closest vector to the target  $t$ .
- ▶ **Learning problems:**
  - ▶ **SIS (Short Integer Solution):** given a matrix  $A$ , find a non-zero  $z$  with  $\|z\| \leq \beta$  s.t.  $Az \equiv 0 \pmod{q}$
  - ▶ **LWE (Learning With Errors):** Given a matrix  $A$  and a vector  $b = As + e$  with  $e$  a small error vector, find  $s$

**Remarks:** learning problems are used in practice but they were proven to be as hard as short/close vector problems, which is why the hardness of CVP and (variants of) SVP matter.

Many variants of CVP and SVP were proven to be NP-hard, hence their cryptographic interest.



## CVP and SVP, a graphical example



## Worst-case to average-case reductions

**Example:** RSA relies on hardness of factoring, but...

- ▶ factoring  $N = 2^k$  is trivial from the binary decomposition of  $N$
- ▶ factoring  $N = p \times \text{NextPrime}(p)$  is easy (because  $p \simeq \sqrt{N}$ )

**Warning:** when a problem is hard, do we talk about the worst case (hardest instance), the average case (random instance) or all cases?

## Worst-case to average-case reductions

**Example:** RSA relies on hardness of factoring, but...

- ▶ factoring  $N = 2^k$  is trivial from the binary decomposition of  $N$
- ▶ factoring  $N = p \times \text{NextPrime}(p)$  is easy (because  $p \simeq \sqrt{N}$ )

**Warning:** when a problem is hard, do we talk about the worst case (hardest instance), the average case (random instance) or all cases?

**Good news:** lattice problems (SIS, LWE) have worst-to-average case reduction properties, i.e. a random instance is expected to be as hard as a worst-case instance.

**Caveat:** those reductions are asymptotic in nature, they provide arguments but do not say anything on cryptographic parameters.

# Kyber, a lattice based cryptosystem

Public key  $(A, t)$

Message  $m$



Alice



Bob

Private key

$$e, s \text{ s.t. } t = As + e$$

$A$  is a matrix,  $e$  a random error

# Kyber, a lattice based cryptosystem

Public key  $(A, t)$

Message  $m$

Small  $e_1, e_2$



Alice



Bob

Private key

$e, s$  s.t.  
 $t = As + e$

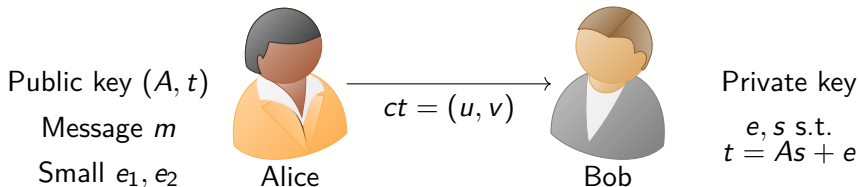
$A$  is a matrix,  $e$  a random error

Encryption = generate random  $r$  and small  $e_1, e_2$  (e.g. coefs  $\in [-\eta, \eta]$ ),

Compute  $u = A^T r + e_1$  and  $v = t^T r + e_2 + \lceil \frac{q}{2} \rceil m$

Decryption = kills  $r$  and error via  $m = \text{Round}(v - s^T u)$

## Kyber, a lattice based cryptosystem



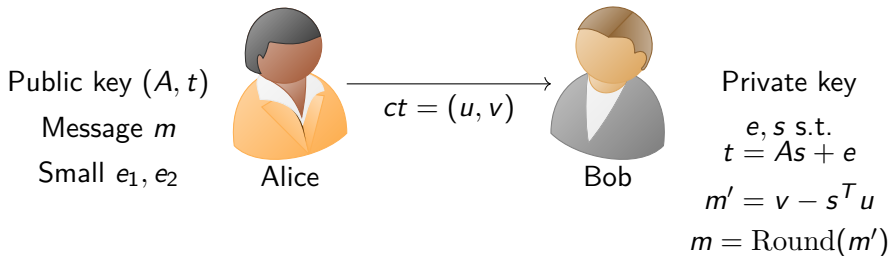
$A$  is a matrix,  $e$  a random error

Encryption = generate random  $r$  and small  $e_1, e_2$  (e.g. coefs  $\in [-\eta, \eta]$ ),

Compute  $u = A^T r + e_1$  and  $v = t^T r + e_2 + \lceil \frac{q}{2} \rceil m$

Decryption = kills  $r$  and error via  $m = \text{Round}(v - s^T u)$

## Kyber, a lattice based cryptosystem



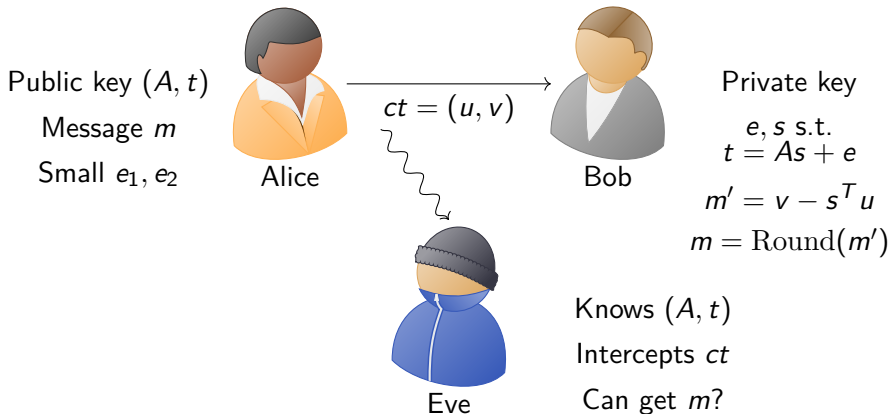
$A$  is a matrix,  $e$  a random error

Encryption = generate random  $r$  and small  $e_1, e_2$  (e.g. coefs  $\in [-\eta, \eta]$ ),

Compute  $u = A^T r + e_1$  and  $v = t^T r + e_2 + \lceil \frac{q}{2} \rceil m$

Decryption = kills  $r$  and error via  $m = \text{Round}(v - s^T u)$

# Kyber, a lattice based cryptosystem



$A$  is a matrix,  $e$  a random error

Encryption = generate random  $r$  and small  $e_1, e_2$  (e.g. coefs  $\in [-\eta, \eta]$ ),

Compute  $u = A^T r + e_1$  and  $v = t^T r + e_2 + \lceil \frac{q}{2} \rceil m$

Decryption = kills  $r$  and error via  $m = \text{Round}(v - s^T u)$

Hardness of module LWE  $\Rightarrow$  only Bob can recover  $m$



# Why it should be familiar

## Kyber's idea:

- ▶ Create random  $r$
- ▶  $u = A^T r + e_1$  bears info on  $r$
- ▶  $v = t^T r + e_2 + \lceil \frac{q}{2} \rceil m$  has both  $r$  and  $m$
- ▶ Knowledge of  $s$  allows to eliminate  $r$  and get  $m$

# Why it should be familiar

## Kyber's idea:

- ▶ Create random  $r$
- ▶  $u = A^T r + e_1$  bears info on  $r$
- ▶  $v = t^T r + e_2 + \lceil \frac{q}{2} \rceil m$  has both  $r$  and  $m$
- ▶ Knowledge of  $s$  allows to eliminate  $r$  and get  $m$

**A classical design:** This was done already by El Gamal!

$ct = (c_1, c_2) = (g^r, mh^r)$ , with  $r$  random and  $h$  is the public key.

This is also done in HQC with  $h$  disappearing from  $v - uy$ .

**What about  $e_2$ ?** It is an optimization that allows to encrypt longer messages without increasing the size of  $A$  or  $t$ .

# Food for thought: lattices and codes

## Lattice-based schemes

LWE: encapsulation  $ct =$

$$(A^T r + e_1, t^T r + e_2 + \lceil q/2 \rceil m)$$

Attack: get  $r$  from  $A^T r + e_1$

## Code-based schemes

Encapsulation  $ct = mG + e$

Decaps:  $m =$  codeword closest to  $ct$

**Question:** are lattices and code the same?

**Distance:** Euclidean (lattices) vs Hamming (codes).

# Food for thought: lattices and codes

## Lattice-based schemes

LWE: encapsulation  $ct = (A^T r + e_1, t^T r + e_2 + \lceil q/2 \rceil m)$   
Attack: get  $r$  from  $A^T r + e_1$

## Code-based schemes

Encapsulation  $ct = mG + e$   
Decaps:  $m = \text{codeword closest to } ct$

**Question:** are lattices and code the same?

**Distance:** Euclidean (lattices) vs Hamming (codes).

These metrics are very different but there is some theoretical work on codes using lattice-based theory. One result coming from this is a reduction from finding short codewords to decoding.

(i.e. if I know how to decode I can find short codewords)

**Idea:** Adapt an equivalent proof reducing from SVP to CVP.

## A quick word on formal notions of security

**EUFCMA:** Existential UnForgeability under Chosen-Message Attacks

A signature is secure if you can query many signatures from chosen messages and yet fail to create a legitimate signature for another message.

**INDCPA/CCA:** INDistinguishability under Chosen Plaintext / Ciphertext Attacks

A PKE/KEM is secure if you can query many encryptions (and decryptions for CCA) and yet fail to distinguish whether a ciphertext comes from one or another message.

EUFCMA is the gold standard for signatures, and IND-CCA is the gold standard for KEM/PKE.

# Real-world cryptography, KEMs

Kyber and HQC's specs are more complicated than these slides!

**PKE vs KEM:** specs are for KEMs yet they use PKE.

## Public key encryption:

- ▶  $\text{Keygen}(alea) = (pk, sk)$
- ▶  $\text{Encrypt}(pk, m) = c$
- ▶  $\text{Decrypt}(sk, c) = m$

## Key encapsulation:

- ▶  $\text{Keygen}(alea) = (ek, dk)$
- ▶  $\text{Encaps}(ek, alea') = (ss, ct)$
- ▶  $\text{Decaps}(dk, ct) = ss$

Encapsulation returns both a ciphertext and a session key  $ss$ .

# Real-world cryptography, KEMs

Kyber and HQC's specs are more complicated than these slides!

**PKE vs KEM:** specs are for KEMs yet they use PKE.

## Public key encryption:

- ▶  $\text{Keygen}(alea) = (pk, sk)$
- ▶  $\text{Encrypt}(pk, m) = c$
- ▶  $\text{Decrypt}(sk, c) = m$

## Key encapsulation:

- ▶  $\text{Keygen}(alea) = (ek, dk)$
- ▶  $\text{Encaps}(ek, alea') = (ss, ct)$
- ▶  $\text{Decaps}(dk, ct) = ss$

Encapsulation returns both a ciphertext and a session key  $ss$ .

**PKE to KEM:** Using a PKE to encrypt a random message  $m$  yields a KEM ( $ss = m, ct = c$ ).

In practice we do it in a more sophisticated way using variants of the so-called **Fujisaki-Okamoto transform**.

# Real-world cryptography, FO transform

**Fujisaki-Okamoto transform:** a generic way to turn an IND-CPA PKE into an IND-CCA KEM, by introducing hash functions.

## How it works:

- ▶ Derandomization: random values (e.g.  $r, e_1, e_2$ ) obtained through hash of (random)  $m$  and public key.
- ▶ Re-encryption: if  $m' = \text{Decaps}(dk, ct)$ ,  $m'$  is re-encrypted to check if  $\text{Encaps}(ek, m') = (ss, ct)$ . If not, Decaps returns a “random” value obtained through hashing (implicit rejection).

**Consequence:** if an attacker sends an invalid ciphertext, the Decaps function will only return a random string, without explicitly returning a failure.

**Caveat:** Re-encryption can also open the door for new attacks, especially side-channel attacks (e.g. timing attacks).



## Real-world cryptography, decapsulation failure

**Decapsulation failure:** when  $ct$  obtained from Encaps differs from  $ct$  after re-encryption, leading Decaps to return the wrong  $ss$ .

**Is this possible? Why and when?**

- ▶ Kyber: if sum of errors  $> q/4$ ,  $\text{Round}(v - s^T u) \neq \lceil q/2 \rceil m$
- ▶ HQC/BIKE: decode can only handle limited number of errors

# Real-world cryptography, decapsulation failure

**Decapsulation failure:** when  $ct$  obtained from Encaps differs from  $ct$  after re-encryption, leading Decaps to return the wrong  $ss$ .

**Is this possible? Why and when?**

- ▶ Kyber: if sum of errors  $> q/4$ ,  $\text{Round}(v - s^T u) \neq \lceil q/2 \rceil m$
- ▶ HQC/BIKE: decode can only handle limited number of errors

**Is it a problem?**

- ▶ **Yes:** on some code-based schemes, there are key-recovery attacks using decapsulation failures.
- ▶ **No:** such failures are proven to be extremely unlikely. Also, using ephemeral keys thwarts failure attacks.
- ▶ **Trivia:** for BIKE and HQC (both QC-MDPC), extensive work on decoding failure rate (DFR), HQC was selected partly because its DFR was better understood than BIKE's.

# An essential operation: polynomial multiplication

At the heart of lattice and code based systems, we find polynomial multiplications in the ring  $R = \mathbb{Z}_q[X]/(X^n \pm 1)$ .

This operation must be **fast**, but also **secure** as we handle secret data (encapsulated messages or private keys).

**First trick: evaluation-interpolation to compute  $PQ(X)$**

- ▶ Evaluate  $P$  and  $Q$  on enough points  $x_i \in \mathbb{Z}_q$
- ▶ Compute all values  $P(x_i)Q(x_i)$  (still in  $\mathbb{Z}_q$ )
- ▶ Interpolate to recover  $PQ$  (in  $R$ )

**Advantage:** we transform operations in  $R$  into operations in  $\mathbb{Z}_q$ .

**Drawback:** how to evaluate and interpolate easily?

**Idea:**  $X^n \pm 1$  yields a wise choice for the  $x_i$ 's.

# An analogue of Fourier transform

**Fourier:** express functions as series in  $\omega = e^{\frac{2i\pi}{n}}$ , i.e.  $\omega^n = 1$ .

**NTT:** evaluation points are  $\psi^k$ , where  $\psi^n = \mp 1 \bmod q$ .

Thanks to that, evaluation is just a matrix product.

**Example:** consider  $q = 7681$ ,  $n = 4$  and

$G(X) = 1 + 2X + 3X^2 + 4X^3$ , represented by  $(1, 2, 3, 4)^T$

A suitable  $\psi$  is either  $\psi = 1925$  or  $\psi = 5956$ .

Evaluating  $G$  on powers of  $\psi$  amounts to computing

$$\hat{g} = \begin{pmatrix} \psi^0 & \psi^1 & \psi^2 & \psi^3 \\ \psi^0 & \psi^3 & \psi^6 & \psi^9 \\ \psi^0 & \psi^5 & \psi^{10} & \psi^{15} \\ \psi^0 & \psi^7 & \psi^{14} & \psi^{21} \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix} = \begin{pmatrix} \psi^0 & \psi^1 & \psi^2 & \psi^3 \\ \psi^0 & \psi^3 & \psi^6 & \psi^1 \\ \psi^0 & \psi^5 & \psi^2 & \psi^7 \\ \psi^0 & \psi^7 & \psi^6 & \psi^5 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}$$

This comes from evaluating  $G$  at  $\psi, \psi^3, \psi^5, \psi^7$  then reducing by  $\psi^8 = \psi^{2n} = 1$ .

# Number Theoretic transform (NTT)

**More ideas:** symmetry  $\psi^{k+n} = -\psi^k$  and periodicity  $\psi^{k+2n} = \psi^k$

$$\hat{g} = \begin{pmatrix} \psi^0 & \psi^1 & \psi^2 & \psi^3 \\ \psi^0 & \psi^3 & -\psi^2 & \psi^1 \\ \psi^0 & -\psi^1 & \psi^2 & -\psi^3 \\ \psi^0 & -\psi^3 & -\psi^2 & \psi^1 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}$$

**Consequence:** only small powers of  $\psi$  involved + nice pattern (butterflies)  $\rightsquigarrow$  efficient implementation.

**Interpolation:** getting  $g$  from  $\hat{g}$  in a similar fashion.

**Conclusion:** we have something fast, but is it secure?

# NTT butterflies

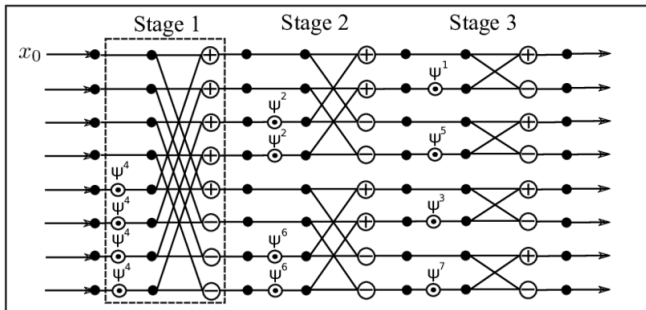


Figure: NTT butterflies for  $n = 8$

This may seem complicated but it is super efficient especially using AVX instructions or hardware accelerators.

# Side-channel attacks

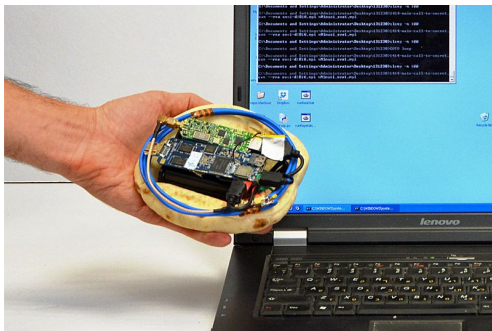
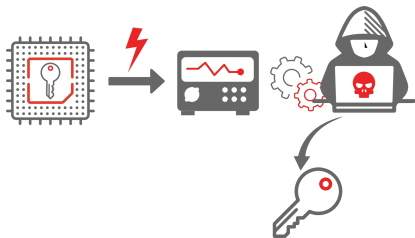


Figure: Portable Instrument for Trace Acquisition (PITA)

# Generalities on side-channel attacks

## The concept of side channels

Measuring physical quantities (EM radiation, power consumption, heat, etc.) during cryptographic computations can leak secrets.



**Figure:** The principle (Source: Secure IC)

**Remark:** does not always require physical access to device (e.g. timing attacks, decryption failure oracle).



# Topics and trends

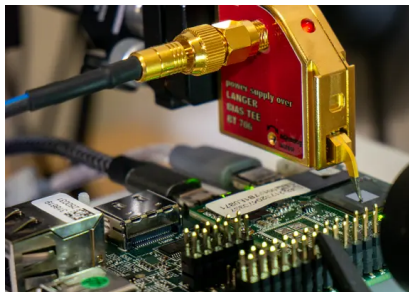


Figure: A probe used for SCA analysis

- ▶ Very active topic of research (more than cryptanalysis)
- ▶ Many opportunities (previous implementations were not designed for side-channel resistance)
- ▶ Efficient tools for acquisition (probes) and exploitation (AI)

## A naive example

Consider discrete logarithm cryptosystems. Main operation is scalar multiplication (ECDLP) or exponentiation (FFDLP).

**Optimization:** use double-and-add strategies to compute  $Q = nP$

- ▶ if  $i$ th bit of  $n$  is 1  $Q := Q + P$
- ▶ if  $i$ th bit is 0  $Q := 2Q$

**Problem:** if one can distinguish doubling from adding (e.g. doubling is usually faster), then one recovers the binary decomposition of the secret  $n$ .

**Remark:** also applies to Euclid's algorithm for GCD computation (many conditional branches). Why it matters? Euclid could be used to compute  $h_0^{-1}$  in BIKE/HQC.

## A nice workaround

**Quick fix:** always compute **both**  $Q + P$  and  $2Q$  and discard one of them. Safe but not very efficient...

### Montgomery ladder:

- ▶ Set  $R_0 = 0$ ,  $R_1 = P$
- ▶ If  $i$ th bit is 0 then  $R_0 := 2R_0$  and  $R_1 := R_1 + R_0$
- ▶ If  $i$ th bit is 1 then  $R_1 := 2R_1$  and  $R_0 := R_1 + R_0$
- ▶ Correctness:  $R_1 = R_0 + P$  (loop invariant)

**Remark:** the if clause depends on  $n$  but each step = adding and doubling regardless of  $n$ .

**Why it's nice?** A **software** fix and no performance cost.

# A taxonomy

**Note:** SCA are often ad-hoc (and implementation dependent) and comes in a huge variety. They are often classified based on

- ▶ their target:
  - ▶ signal: sound, power, EM
  - ▶ timing attacks
  - ▶ cache/memory attacks
  - ▶ ... and many more
- ▶ the attacker's resources:
  - ▶ single-trace attacks (one chance)
  - ▶ multi-trace attacks (multiple attempts)
  - ▶ template attacks (bought model device)
  - ▶ fault injection attacks (tampers with device)

# Simple Power Attack (SPA)

**Simple power attack (SPA):** using one or a few traces, an attacker recovers the secret. Even be doable on the fly without much preparation (depends on noise).

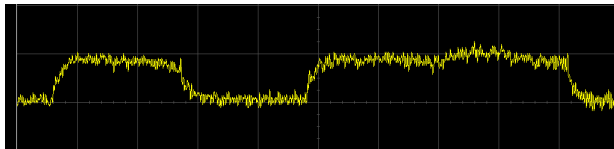


Figure: RSA trace when secret bit is 0 (left) or 1 (right)

# Differential Power Analysis (DPA)

**Step 1:** the attacker has access to the device and first establish models for the power consumption by acquiring many traces.

**Step 2:** These models are used to predict secret information based on the power trace observed.

**Idea:** from the observed signal, find the closest template (maximum likelihood). The input yielding this template should be the same as the one who led to the signal.

**Remark:** This attack is harder to mount but realistic for commercial devices and much more resilient to noise.

## An innocent example

Within several lattice-based KEMs (NewHope, Frodo, etc.), an innocuous function encoding bit strings into vectors/matrices.

$$\text{Bitstring } m \mapsto \text{array } m \text{ s.t. } m[i] = \begin{cases} 0 & \text{if } m_i = 0 \\ \lceil q/2^B \rceil & \text{if } m_i = 1 \end{cases}$$

Innocuous? It parses secret data to be encapsulated!

**Step 1:** execute  $\text{Encode}(m)$  for  $m = 0x00, 0x01, \dots, 0xff$  and gather templates. Acquire the corresponding 256 traces (in practice maybe 100 traces for each).

**Step 2:** when the  $i$ th byte of  $m$  is processed, recover the signal and find the template that fits the observation best.

**Result:** byte by byte, we recover  $m$  or something close enough.

## Countermeasure: shuffling

**Solution:** randomize parsing order.

- ▶ Assume  $m$  is a 256-bit session key.
- ▶ It has  $\simeq 128$  bits equal to 1 ( $m$  is random)
- ▶ There are  $\binom{256}{128}$  keys with Hamming weight 128
- ▶ This is much more than anyone can bruteforce.

**Result:** an attacker no longer know which byte of  $m$  he observed, so at best he gets the Hamming weight of  $m$ .

**Warning:** here it is fine but sometimes the Hamming weight is sensitive! (e.g. for code-based crypto)



# Attacks against NTT

What do I need to compute  $\hat{g} = \text{NTT}(g)$ ?

All  $g_i \times \psi^k$ , for  $g_i \in \mathbb{F}_q$  and  $k \leq n/2$ .

**Idea:** there is only  $nq/2$  options i.e.  $\simeq 10^6$  for Kyber's parameters

**Step 1:** gather signal traces corresponding to each  $(g_i, \psi^k)$

**Step 2:** Capture signal during execution and identify the  $g_i$  whose template matches observation best (maximum likelihood).

**Consequences:** within Kyber the product is used

- ▶ for encryption in  $A^T r$  and  $t^T r \rightsquigarrow$  might leak  $r$  (and thus  $m$ )
- ▶ for decryption in  $s^T u \rightsquigarrow$  might leak the private key  $s$

**Key/message recovery:**  $m$  is often a session key within a protocol  $\rightsquigarrow$  one session compromised.

# Attacks against NTT: targets and options

## Other targets?

- ▶ Dilithium is more challenging, as  $nq/2 \simeq 10^{10}$
- ▶ Code-based crypto (BIKE, HQC) often avoid NTT
- ▶ FrodoKEM does not use the module structure

## Countermeasures:

- ▶ Blinding: shift secret ( $P \mapsto X^\rho \times P$ ) before product and recover product  $X^\rho PQ \mapsto PQ$ . Idea:  $X^\rho \times P$  leaks and not  $P$ . Cheap but not so strong: limited options as  $\rho < n$  ( $X^n = \mp 1$ )
- ▶ Shuffling? Would break butterfly structure
- ▶ Masking? cf. following slides

# Good practices for safe implementations

- ▶ Avoid branches and array access depending on secret values
- ▶ When no other option, at least ensure that both branches yield to same type and number of operations
- ▶ Identify and minimize the code lines where secret is involved
- ▶ Randomize when possible: e.g. in  $\mathbb{F}_p$  replace  $x^n$  by  $x^{n+k(p-1)}$  (blinding using  $x^{p-1} = 1 \bmod p$ .)

## Beyond good practices

Even good practices are not sufficient to prevent intrinsic leakage.  
In practice, several options are considered:

- ▶ **Emission control:** use Faraday cages, measure EM emissions and confirm that they are unusable (e.g. for satellites) or make sure that no attacker can be physically too close to cryptographic devices (NATO TEMPEST)
- ▶ **Parasite emissions:** generate noise, or run tasks in parallel to make it harder to identify the relevant signal
- ▶ **Shuffling & masking:** ensure that leakage is not directly related to secret information

First option has an impact on device cost, last two have an impact on device performance (not unusual to have factors 10 to 100).

# Shuffling

**Idea:** when performing operations on the coordinates of a secret vector  $x$ , for instance to compute  $\sum_i a_i x_i$ , randomize the order in which the  $a_i x_i$ 's are computed and summed.

**Why?** If  $x_i = 0$ ,  $a_i x_i$  is much easier to compute, so an attacker could identify zeroes in  $x$  (thus  $x$  if  $x \in \mathbb{F}_2^n$ ). If the order is randomized, no way of knowing which  $x_i$  is 0.

## Limits:

- ▶ Heavy cost (generating random permutations, deactivating compiler/hardware optimization)
- ▶ Not always sufficient (in the above example, the Hamming weight of  $x$  still leaks)

# Masking

- ▶ **Basic masking:** pick random mask  $m$  and perform operations on  $x \oplus m$  to ensure that leakage is not related to  $x$  anymore. Can also be done with arithmetic operations (Boolean vs arithmetic masking).
- ▶ **ISW masking:** cut the secret  $x$  into slices and XOR the slices with masks. Invented by Ishai, Sahai and Wagner in 2003, current main standard for masking.

## Limits:

- ▶ Heavy cost (even worse than shuffling)
- ▶ Composition of masked primitives is not always secure
- ▶ Security model assume perfect hardware and randomness

# Masking, a small example

**Objective:** Securely compute  $c = ab$ , where  $a = a_1 \oplus a_2 \oplus a_3$  and  $b = b_1 \oplus b_2 \oplus b_3$  are both secret, by computing slices  $c_1, c_2, c_3$  such that  $c = c_1 \oplus c_2 \oplus c_3$ .

**Option 1:** using random  $z_{ij}$

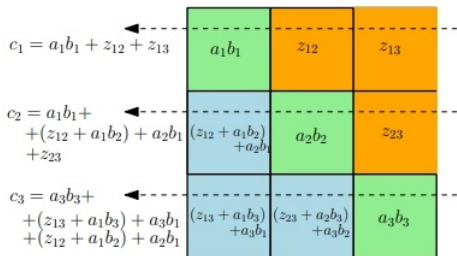
**Option 2:** using threshold

$$c_1 = a_2 b_2 \oplus a_2 b_3 \oplus a_3 b_2$$

$$c_2 = a_1 b_3 \oplus a_3 b_1 \oplus a_3 b_3$$

$$c_3 = a_1 b_1 \oplus a_1 b_2 \oplus a_2 b_1$$

( $c_i$  independent of  $a_i, b_i$ ).



In both cases, one can prove that observing one share  $c_i$  is not enough to completely recover  $a$  or  $b$ .

## Higher order masking

Depending on the model, a  $d$ -order masked implementation refers to:

- ▶ A masked implementation that is secure against order  $d$  power analysis
- ▶ A threshold scheme in which an attacker needs at least  $d + 1$  shares to recover secret values

High-order masking is not an easy task as it incurs a significant computational cost. Some designs were proposed to reduce the cost (e.g. use less randomness) but many failed.

Though masking aims at making side-channel attacks theoretically impossible, in practice even first order masking makes them much harder to perform.



# Masking efforts in practice

## The various flavors of masking:

- ▶ Cheap techniques providing some security at virtually no cost: scalar / base point blinding, projective coordinates with  $Z \neq 1$
- ▶ So-called “gadgets” for limited sensitive operations (e.g. product, sum) provide security at a price.
- ▶ End-to-end masked implementation: assembling gadgets to cover all sensitive parts of an algorithm. Secure but extremely expensive (not sure it is used in practice)

## Current trends in masking:

- ▶ Provide gadgets for asymmetric crypto, especially post-quantum (lattices + BIKE so far)
- ▶ Schemes that designed for easier masking (e.g. Raccoon, an analogue of ML-DSA)

## Beyond SCA, fault-injection attacks

**Taxonomy:** voltage glitches, clock glitches or using devices (lasers, EM fields) to force operations on tampered data.

**Examples:** zero-value attacks, nonce reuse, RowHammer attack on ARM/Android in 2016 (voltage glitch in DRAM cells).

**Limits:** access to device, static keys, expensive equipment.

**Scenarios:** widely spread devices on which we can experiment, or powerful attacker (state), e.g. on recovered military equipment.

**Countermeasures:** expensive hardware countermeasures (e.g. shielding device to prevent tampering, or ensuring that any fault injection attempt destroys the device).

# Conclusion

- ▶ Post-quantum cryptography comes with technical challenges:
  - ▶ Transition  $\rightsquigarrow$  new software & hardware
  - ▶ Performance (mostly bandwidth/fragmentation issue)
  - ▶ Maturity (both algorithms and implementations)
- ▶ Yet the main challenges are political:
  - ▶ Common standards: US vs EU (vs China, India, Korea?)
  - ▶ Transition plan (EU states must have one before 2027)
  - ▶ Investment & training
  - ▶ Define certification criteria

# Beyond cryptography: the world of cybersecurity

Cybersecurity is a complex ecosystem where many constraints and actors have a say. **Cryptography is one aspect among others!**

- ▶ **Key management:** distributing, updating and storing keys is a major challenge involving complex solutions.
- ▶ **Secure software:** even the strongest algorithm is useless if poorly implemented or if there is an interface bug. Such flaws are not easy to detect!
- ▶ **Hardware security:** physical attacks (e.g. arson) against servers, fault injection and side-channel attacks, memory attacks, you name it...
- ▶ **Culture of security:** is the user trained, aware, careful? Cryptography can do nothing if the password is Hell0world.